UPSKILL SERIES · DAY 1A

The Reverse Problem

Python → Go

Basic Data Types — Strings, Slices, Tuples, Maps, Numbers & Stacks/Queues

9 ways. 6 basic data types. Real input → output, side by side.

From a real career transition: Python → Google Go (Golang)

Why Python → Google Go?

Context before the code — here's why I'm making this shift.

Performance & Concurrency

Go is compiled, statically typed, and built for concurrency with goroutines — ideal for backend services that need to scale under real load.

Industry Demand

Go powers Docker, Kubernetes, and countless cloud-native tools. Companies building modern infrastructure are actively hiring for it.

Simplicity by Design

Go has a small, opinionated syntax — fewer ways to do the same thing, which makes large codebases easier to read and maintain.

Complements Python, Doesn't Replace It

Python remains unmatched for scripting, data, and automation. Go adds a second tool for systems-level and high-performance work.

Why Start With "Reverse"?

Not Hello World. A problem that touches every data structure.

1	**Easy to Understand**	Everyone — beginner or expert — instantly gets what "reverse" means. Zero conceptual overhead.
2	**Covers Every Core Data Type**	Strings, slices, maps, numbers, stacks — reverse forces you to touch each one.
3	**Reveals Language Philosophy Fast**	One simple goal, two totally different toolsets — the perfect lens to compare Python and Go.
4	**Builds Real Muscle Memory**	Two-pointer swaps, pointer manipulation, and memory thinking — patterns reused everywhere in Go.
5	**Naturally Extends to Harder Problems**	Today: reverse. Tomorrow: sorting, searching, and beyond — same comparative format.

Why "Reverse"?

One problem. Every core data structure. Two languages, two philosophies.

The Idea

I'm transitioning from Python to Google's Go (Golang) for backend & systems work.

Instead of "Hello World", I picked the classic Reverse problem — and solved it across EVERY basic data type.

Result: 9 distinct techniques, real input/output, and a side-by-side of how each language THINKS.

Today's Data Types

1. String Reversal
2. List / Slice Reversal
3. Tuple / Array Reversal
4. Dictionary / Map Reversal
5. Number / Integer Reversal
6. Stack & Queue Reversal

→ Advanced types (linked list, matrix, tree, bits...) in Day 1b

1 String Reversal

Reversing the order of characters in text

PYTHON

```python
s = "upskill"
print(s[::-1])

# Input  : "upskill"
# Output : "llikspu"
```

How it converts:

Slice notation [::-1] means "start at the end, step backward by 1." Python builds a brand-new string by reading characters from last to first — no loop visible to the developer.

GOOGLE GO

```go
func reverseString(s string) string {
  r := []rune(s)
  for i, j := 0, len(r)-1; i < j; i, j = i+1, j-1 {
    r[i], r[j] = r[j], r[i]
  }
  return string(r)
}

// Input  : "upskill"
// Output : "llikspu"
```

How it converts:

Go converts the string to a slice of runes (Unicode-safe characters), then swaps characters from both ends inward using two pointers — fully explicit and in-place.

KEY INSIGHT: Go uses []rune, not bytes — this matters for Unicode/emoji safety. More on this in Day 3 (Edge Cases).

2 List / Slice Reversal

Reversing the order of elements in a collection

PYTHON

```python
lst = [10, 20, 30, 40, 50]
print(lst[::-1])

# Input  : [10, 20, 30, 40, 50]
# Output : [50, 40, 30, 20, 10]
```

How it converts:

The same [::-1] slicing trick works on lists. Python creates a new list, copying elements starting from the last index back to the first — concise but allocates new memory.

GOOGLE GO

```go
func reverseSlice(s []int) []int {
  for i, j := 0, len(s)-1; i < j; i, j = i+1, j-1 {
    s[i], s[j] = s[j], s[i]
  }
  return s
}

// Input  : []int{10,20,30,40,50}
// Output : []int{50,40,30,20,10}
```

How it converts:

Same two-pointer swap as strings. No new memory is allocated — the original slice is reversed in-place, which is faster and more memory-efficient for large datasets.

KEY INSIGHT: Python's slicing is elegant but copies data. Go's in-place swap is the pattern you'll reuse for almost every reversal problem.

3 Tuple / Array Reversal

Reversing fixed-size, immutable-style collections

PYTHON

```python
t = (1, 2, 3, 4)
print(tuple(reversed(t)))

# Input  : (1, 2, 3, 4)
# Output : (4, 3, 2, 1)
```

How it converts:

Tuples are immutable, so you can't reverse in-place. reversed() returns an iterator that walks backward, and tuple() rebuilds a brand-new tuple from it.

GOOGLE GO

```go
func reverseArray(a [4]int) [4]int {
  for i, j := 0, len(a)-1; i < j; i, j = i+1, j-1 {
    a[i], a[j] = a[j], a[i]
  }
  return a
}

// Input  : [4]int{1, 2, 3, 4}
// Output : [4]int{4, 3, 2, 1}
```

How it converts:

Go has NO tuple type at all. The closest equivalent is a fixed-size array, which is fully mutable — so it gets the exact same two-pointer swap as a slice.

KEY INSIGHT: Python protects you from accidental mutation (immutability). Go gives you direct memory control instead — a tradeoff of safety vs. performance.

4 Dictionary / Map Reversal

Reversing key-value pairs (and the ordering problem)

PYTHON

```
d = {"a": 1, "b": 2, "c": 3}
print(dict(reversed(d.items())))

# Input  : {"a":1, "b":2, "c":3}
# Output : {"c":3, "b":2, "a":1}
```

How it converts:

Since Python 3.7, dicts preserve insertion order. reversed(d.items()) walks key-value pairs backward, and dict() rebuilds the dictionary in that reversed order.

GOOGLE GO

```
keys := []string{"a", "b", "c"}
for i, j := 0, len(keys)-1; i < j; i, j = i+1, j-1 {
  keys[i], keys[j] = keys[j], keys[i]
}

// Input  : keys = ["a","b","c"]
// Output : keys = ["c","b","a"]
```

How it converts:

Go's map has NO ordering guarantee at all — even iteration order is randomized. To "reverse" anything, you must track keys in a separate slice and reverse that.

KEY INSIGHT: This is the BIGGEST mindset shift. Python hides ordering complexity; Go forces you to design your own ordering layer from day one.

5 Number / Integer Reversal

Reversing digits of a number (including negatives)

PYTHON

```
n = -1234
sign = -1 if n < 0 else 1
print(sign * int(str(abs(n))[::-1]))

# Input  : -1234
# Output : -4321
```

How it converts:

Strip the sign, convert the absolute number to a string, reverse that string with slicing, convert back to an integer, then reapply the original sign.

GOOGLE GO

```
func reverseNumber(n int) int {
  sign := 1
  if n < 0 { sign = -1; n = -n }
  result := 0
  for n != 0 {
    result = result*10 + n%10
    n /= 10
  }
  return sign * result
}
// Input: -1234   Output: -4321
```

How it converts:

No string conversion at all — pure math. Repeatedly extract the last digit with % 10, shift the result left by multiplying by 10, then strip the digit with integer division.

KEY INSIGHT: Python reaches for strings as a universal tool. Go's math-first approach avoids allocations entirely — faster, and watch for int overflow at scale.

6 Stack & Queue Reversal

Reversing LIFO and FIFO structures

PYTHON

```python
# Stack
stack = [1, 2, 3, 4]
print(stack[::-1])
# Output: [4, 3, 2, 1]

# Queue (deque)
from collections import deque
q = deque([1, 2, 3, 4])
q.reverse()
print(q)
# Output: deque([4, 3, 2, 1])
```

How it converts:

Stacks reuse list slicing. Queues use collections.deque, whose .reverse() flips the doubly-linked structure in-place — both are one-liners.

GOOGLE GO

```go
// Both stack & queue use []int
func reverseSeq(s []int) []int {
  for i, j := 0, len(s)-1; i < j; i, j = i+1, j-1 {
    s[i], s[j] = s[j], s[i]
  }
  return s
}

// Input  : []int{1, 2, 3, 4}
// Output : []int{4, 3, 2, 1}
```

How it converts:

Go has no built-in stack, queue, or deque type. Slices serve double duty for both — the SAME two-pointer swap function handles either case.

KEY INSIGHT: Python gives you a specialized tool for every job (deque, list). Go gives you ONE tool (slice) that you adapt to every job.

What I Learned — 6 Types In

Same problem, two completely different mental models.

1 Python optimizes for developer time

Built-in slicing [::-1], reversed(), and deque give one-line solutions for almost everything.

2 Go optimizes for runtime control

Two-pointer in-place swaps repeat across string, slice, array & stack — one pattern, many uses.

3 Go has fewer specialized types

No tuples, no deque, no ordered maps — slices and explicit loops cover most cases.

4 Ordering is explicit in Go

Python's dict keeps insertion order automatically. Go's map has NONE — you track order yourself.

Where "Reverse" Shows Up in Real Systems

This isn't just an interview question — it's everywhere in production code.

Palindrome Checks

Input validation, DNA sequence analysis, data integrity checks

Undo / Redo Stacks

Text editors, design tools, version history navigation

Browser History

Back-button navigation, in-app navigation stacks

Reverse Log Iteration

Showing most-recent events first in feeds & monitoring dashboards

Number Processing

Checksum validation, digit-based algorithms, hashing utilities

Map/Dict Reordering

Caching strategies (LRU), config precedence, priority resolution

Coming Up Next

Day 1b: Advanced Data Types

Linked Lists, Matrices, Trees, Bit Manipulation, Sentences & File Streams — the algorithms that separate scripting from systems engineering.

DAY 2

Testing the Reverse Functions

pytest vs Go's built-in testing package — same scenario, same assertions, two philosophies on test design.

DAY 3

Edge Cases & Breaking Things

Empty inputs, Unicode/emoji, nil pointers, integer overflow — the bugs that separate junior from senior code.

Follow along — full code for every example will be linked in the comments.